Report

Team 15
Han Wang, Xiang Liu

Problem 1. Joint architecture

1.1 Hyper-parameter tuning

(a) Optimizer and learning rate

1. What is the best choice for the optimizer(SGD vs. Adam)?

Adam.

2. The optimal learning rate for both SGD and Adam?

SGD: 0.01.

Adam: 0.0001.

3. How does the learning rate affect the learning process?

Typically, the learning rate cannot be so low or so high. If the learning rate set is too low, the training will progress slowly due to the tiny update of the weights. If the learning rate set is too high, the network may fail to converge and just hover around the optimal value. As for Adam, the performance gets better with the decreasing of learning rate.(figure below)

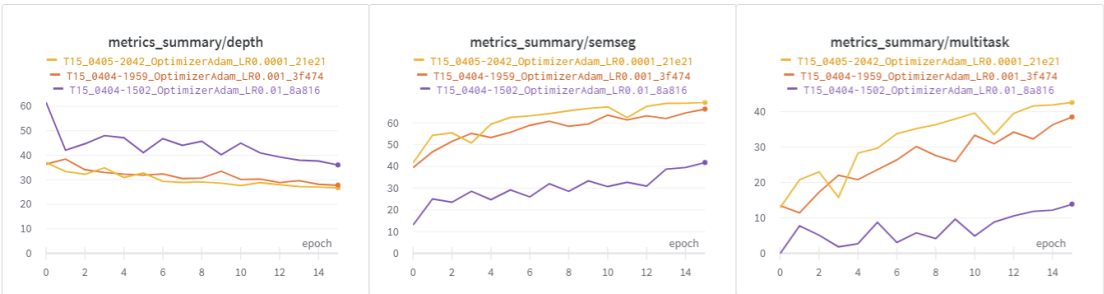

Figure 1: performance of Adam with different learning rate

As for SGD, the performance gets worse with the decreasing of the learning rate because of the slow convergence. SGD is prone to track in local minima, as no second moment information is utilized when performing gradient descent. Thus, in this case, SGD with larger learning rate may converge faster to the local minima.

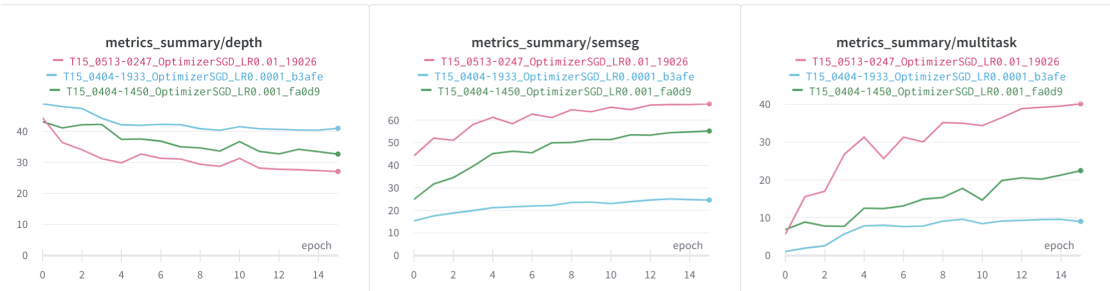

Figure2: performance of SGD with different learning rate

4. How does using SGD compare to Adam?

In this exercise, using Adam performs better than using SGD when the batch size and learning rate are set the same. Figure below demonstrates it when the learning rate is set 0.001 and 0.0001 respectively. In theory, Adam stores an exponentially decaying average

of past squared gradients, both the first moment and the second moment to get an adaptive learning rate, which demonstrates good properties on large datasets.

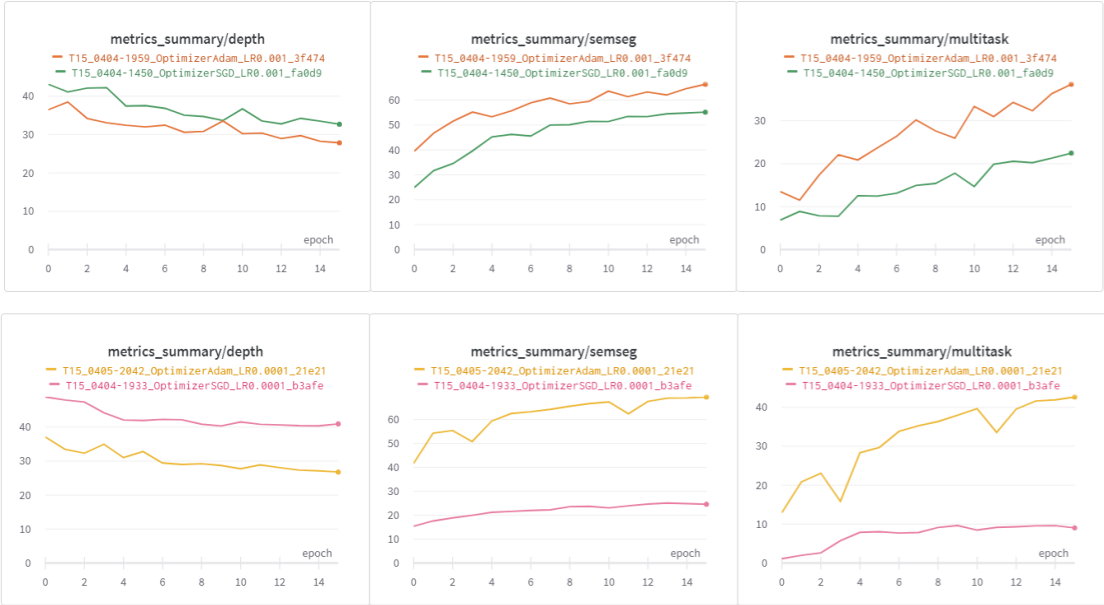

Figure 3: performance comparision of Adam and SGD with the same learning rate

However, SGD could perform better and Adam may have some problems if leaving the current scene aside. For example, Adam may fail to convergence[1] or miss the global optimal solution[2].

(b) Batch size

1. How does the batch size affect the performance metrics of the multitask network?

At the same epoch, a small batch size of 4 performs the best of a batch size of 8 and 16. It takes more epochs to achieve similar metrics for a larger batch size.

The table below shows the metrics of different batch size at different epoch and their final metrics.

Table 1: performance of different batch size

epoch \ batch size	4	8	16
5	**29.712**	25.679	28.955
10	**39.644**	36.977	33.829
15	**42.652**	40.403	36.275
Last	42.652	46.264	**48.068**

2. Which other practically relevant aspects besides the task metrics does the batch size affect and how?

In terms of the **validation loss**, the larger the batch size, the larger the validation loss at the same epoch and the more epochs it takes to converge to the minimum validation

[1] Reddi S J, Kale S, Kumar S. On the convergence of adam and beyond[J]. arXiv preprint arXiv:1904.09237, 2019.
[2] Keskar N S, Socher R. Improving generalization performance by switching from adam to sgd[J]. arXiv preprint arXiv:1712.07628, 2017.

loss.(figure below) This is because a small batch size can introduce noise and lead to a better generalization. Also, Training with mini-batches tends to converge to a flat minimization that varies only slightly within the small neighborhood of the minimization, whereas large-batch converges to a sharp minimization, which varies greatly. Flat minimizers tend to generalize better because they are more robust to changes between training and test sets[3].

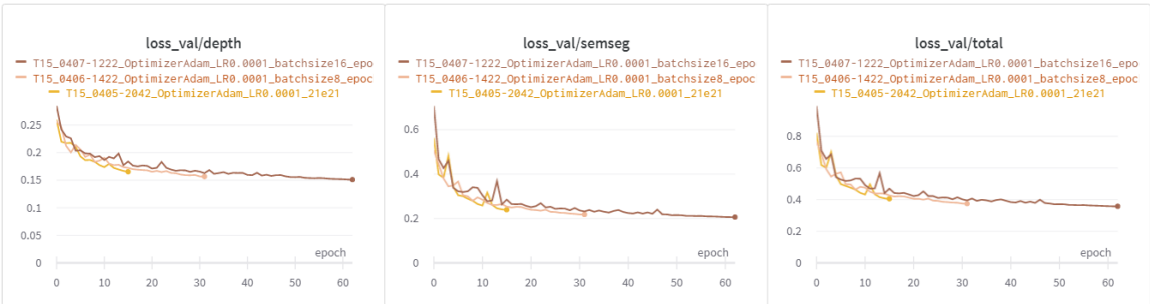

Figure 4: validation loss with different batch size

In terms of **GPU memory**, less batch size, and therefore less epochs, leads to less GPU memory allocated and faster computation.

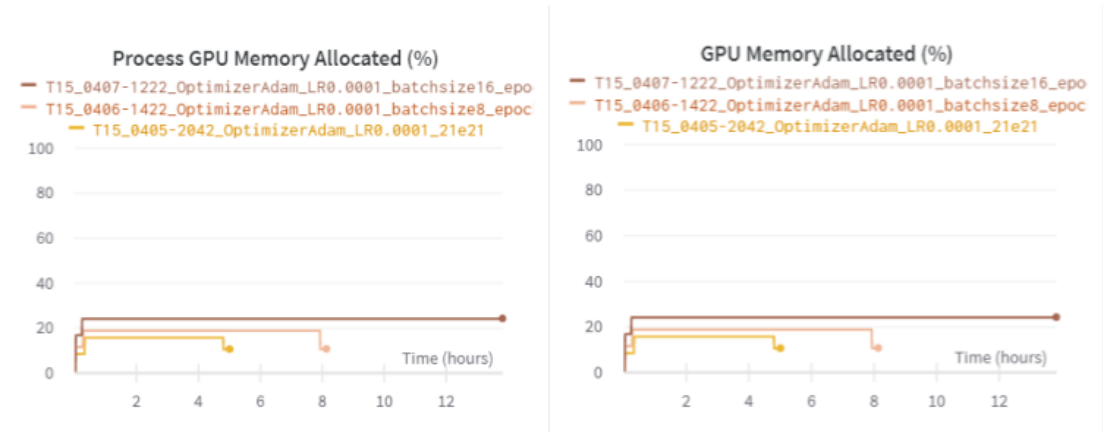

Figure 5: GPU memory with different batch size

(c) Task weighting

1. **What is the effect of increasing the loss weight for one task while decreasing the weight for the other task?**

If increasing the loss weight for one task, the performance of this task will get better and the performance of the other task will get worse.

The diagram below shows the metrics when the weights of the segmentation are 0.3(red), 0.5(yellow), and 0.7 (blue) respectively. We can see that the metrics of semantic segmentation increase with the increasing of segmentation weights, while the metric of depth does not.

[3] Keskar N S, Mudigere D, Nocedal J, et al. On large-batch training for deep learning: Generalization gap and sharp minima[J]. arXiv preprint arXiv:1609.04836, 2016.

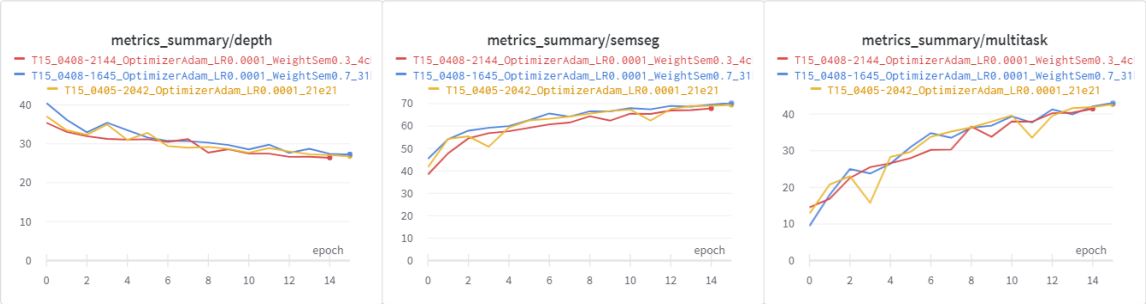

Figure 6: performance with different task weight

2. **How should the task weights be chosen and why?**

Simply performing a weighted linear sum of the losses for each individual task is a naive approach, which is expensive to tune. We need to jointly consider the performance of all the tasks like this task.

In addition, there exists other methods to avoid expensive tuning. For example, design a network to learn optimal task weightings using ideas from probabilistic modelling[4]. Specifically, use homoscedastic uncertainty as a basis for weighting losses in a multi-task problem.

1.2 Hardcoded hyperparameters

(a) Encoder initialization

1. How are the weights of the encoder network initialized in the solution template?

Pytorch provides many pre-trained models for different tasks including semantic segmentation. Take DeepLabV3 ResNet as an example, the pre-trained models have been trained on a subset of COCO train2017, on the 20 categories that are present in the Pascal VOC dataset[5].

2. What happens when you switch the pretrained flag of the encoder?

When switching the pretrained flag to TRUE, the performance of the multi-task model becomes better.

3. How do both variants compare performance-wise and what might be the reason for that?

The following diagram shows that using a pretrained model(green) can improve the performance. The metrics of segmentation and multi-task get higher and the metric of depth gets lower especially at the beginning of the train.

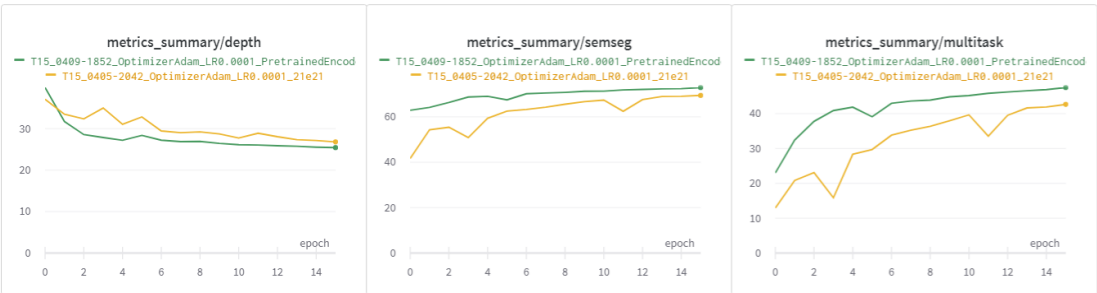

Figure 7: performance comparison of pretrained backbone

[4] Kendall A, Gal Y, Cipolla R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 7482-7491.

[5] https://pytorch.org/vision/stable/models.html#deeplabv3

When directly analyzing the prediction image, the prediction of the model pretrained(right) is able to identify road signs but the model without pretrained(left) cannot. From the depth prediction perspective, the pretrained model is able to predict the depth of the person more precisely.

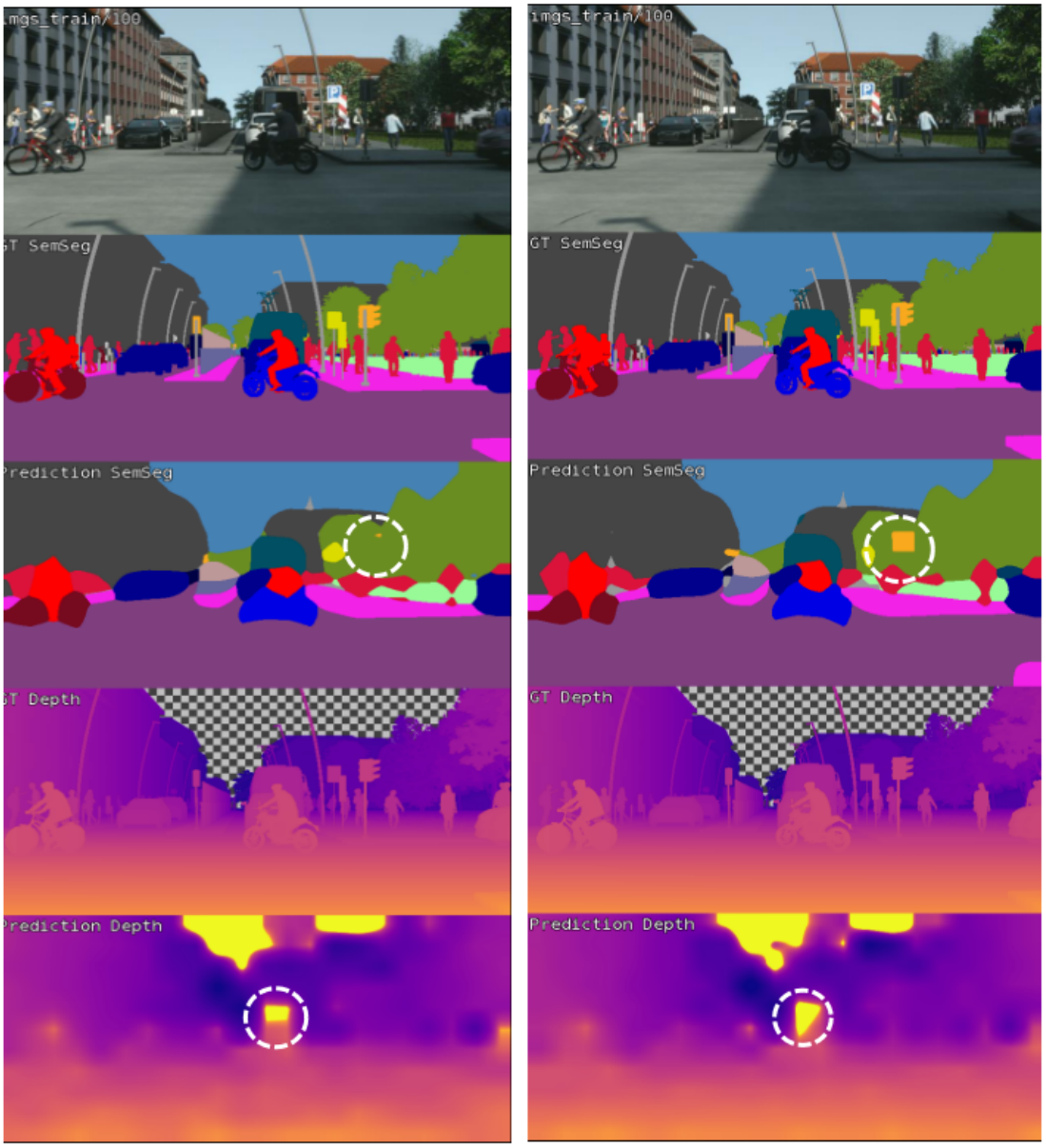

Figure 8: prediction results comparison of pretrained backbone

The idea of pre-trained models is similar to transfer learning. For image-based tasks, using pre-trained models is able to capture better features to improve the performance of the following model.

(b) Dilated convolutions

1. Set dilation flags to (False, False, True) and train the model. Does the performance improve? If so, why?

Yes, the performance gets better when we set dilation flags to (False, False, True).
It is clear that using dilated convolutions(purple) greatly improves the performance.

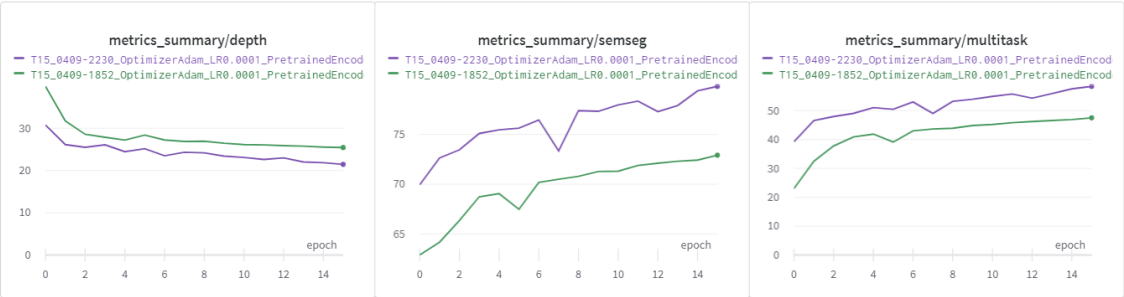

Figure 9: performance comparison of dilated convolution

Again from the prediction result, the right model with dilated convolutions clearly identifies the triangle street sign and its pole. And the depth prediction image as a whole becomes clearer.

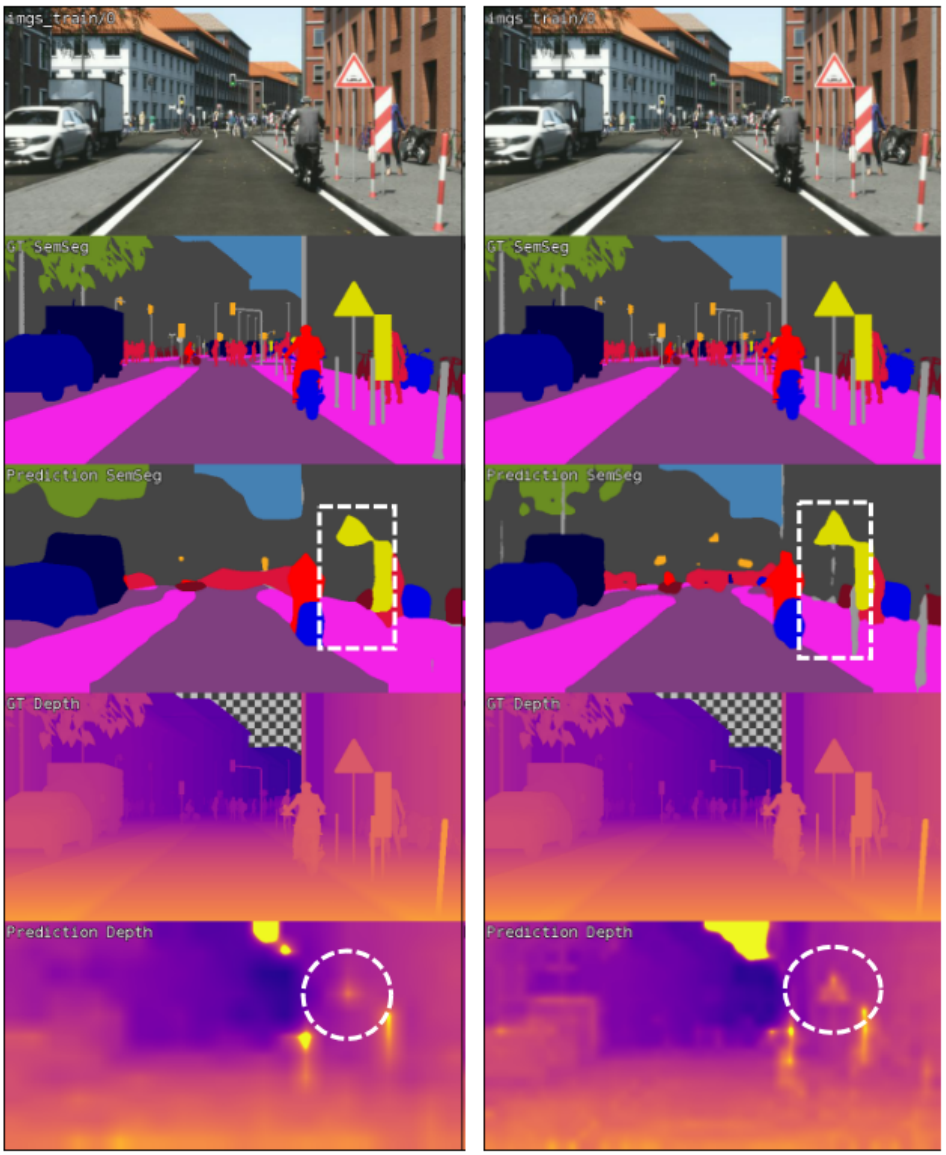

Figure 10: prediction results comparison of dilated convolution

Firstly, dilated convolutions are helpful for pixel-wise tasks like semantic segmentation, because it solves the problem of resolution reduction and information loss caused by downsampling. We can adaptively modify the filter's field-of-view by changing the rate value to capture multi-scale information.

1.3 ASPP and skip connections

What is the intention of adding ASPP and skip connections? How does the model performance change with ASPP and skip connections functioning?

Adding ASPP encodes multi-scale contextual information. Adding skip connections generally solves the degradation problem, which means deep layers often fail to learn even identity mappings. So skip connections enable us to make full use of the features of lower layers.

As shown in Table 2, the multi-task metrics improves significantly, which is mainly attributed to the surge in semantic segmentation. It can be induced that incorporating multi-scale and low-level features will largely boost the performance of dense prediction task like semantic segmentation, which can instantiated by the finer prediction result shown in Figure 11.

Table 2: performance comparison of ASPP and skip connections

	multi-task metrics↑	**semantic segmentation metrics↑**	**depth metrics↓**
Default	58.388	79.824	21.436
ASPP-skip	**64.855**	**84.641**	**19.786**

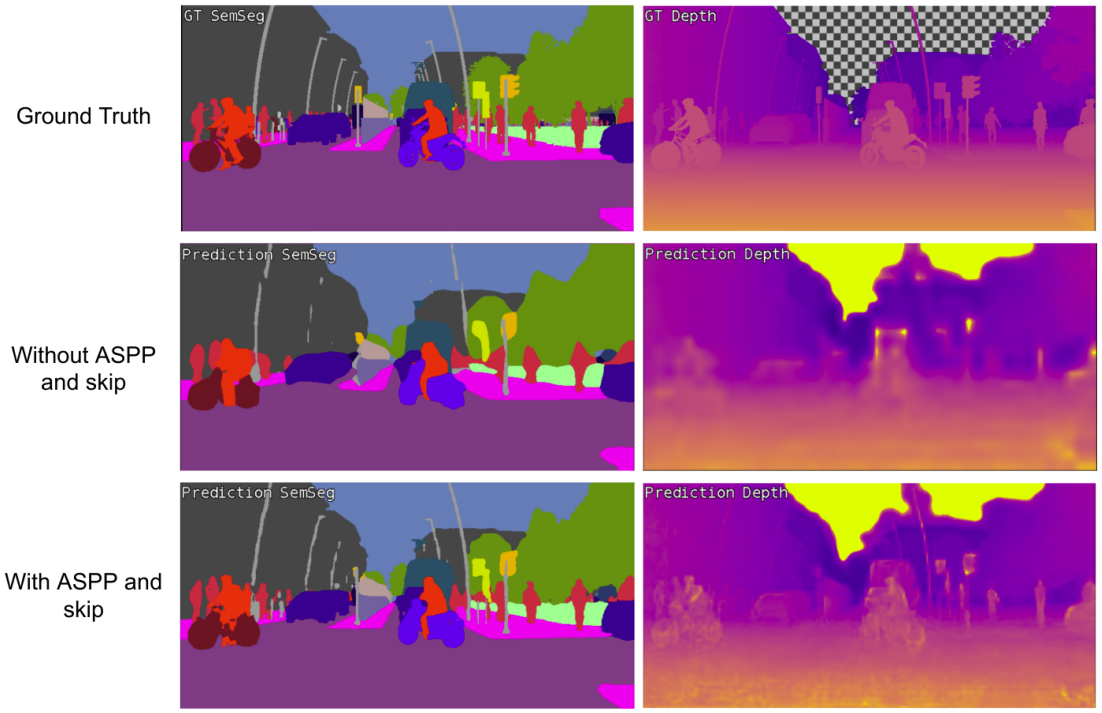

Figure 11: prediction results comparison of ASPP and skip connections

Code snippets of decoder with skip connection and ASPP are shown below.

```
class DecoderDeeplabV3p(torch.nn.Module):
   def __init__(self, bottleneck_ch, skip_4x_ch, num_out_ch):
       super(DecoderDeeplabV3p, self).__init__()
```

```
        # TODO: Implement a proper decoder with skip connections instead of the following
        self.compress_features = torch.nn.Sequential(
            torch.nn.Conv2d(skip_4x_ch, 48, kernel_size=1, stride=1, bias=False),
            torch.nn.BatchNorm2d(48),
            torch.nn.ReLU(),
        )

        ### padding here
        self.features = torch.nn.Sequential(
            torch.nn.Conv2d(
                48 + bottleneck_ch, 256, kernel_size=3, stride=1, padding=1, bias=False
            ),
            torch.nn.BatchNorm2d(256),
            torch.nn.ReLU(),
            torch.nn.Conv2d(256, 256, kernel_size=3, stride=1, padding=1, bias=False),
            torch.nn.BatchNorm2d(256),
            torch.nn.ReLU(),
        )

        self.features_to_predictions = torch.nn.Conv2d(
            256, num_out_ch, kernel_size=1, stride=1)

    def forward(self, features_bottleneck, features_skip_4x):
        """
        DeepLabV3+ style decoder
        :param features_bottleneck: bottleneck features of scale > 4
        :param features_skip_4x: features of encoder of scale == 4
        :return: features with 256 channels and the final tensor of predictions
        """
        # TODO: Implement a proper decoder with skip connections instead of the following;
keep returned
        #       tensors in the same order and of the same shape.

        features_4x = F.interpolate(
            features_bottleneck,
            size=features_skip_4x.shape[2:],
            mode="bilinear",
            align_corners=False,
        )
        features_skip_4x = self.compress_features(features_skip_4x)
        # not really sure how to make it into 256 channels
        features_4x = torch.cat((features_4x, features_skip_4x), 1)

        features_4x = self.features(features_4x)

        predictions_4x = self.features_to_predictions(features_4x)

        return predictions_4x, features_4x


class ASPP(torch.nn.Module):
    def __init__(self, in_channels, out_channels, rates=(3, 6, 9)):
        super().__init__()
        # TODO: Implement ASPP properly instead of the following
        self.conv_1x1 = ASPPpart(
            in_channels, out_channels, kernel_size=1, stride=1, padding=0, dilation=1
        )
        self.conv_dilated_first = ASPPpart(
            in_channels,
            out_channels,
            kernel_size=3,
            stride=1,
            padding=rates[0],
```

```
        dilation=rates[0],
    )
    self.conv_dilated_second = ASPPpart(
        in_channels,
        out_channels,
        kernel_size=3,
        stride=1,
        padding=rates[1],
        dilation=rates[1],
    )
    self.conv_dilated_third = ASPPpart(
        in_channels,
        out_channels,
        kernel_size=3,
        stride=1,
        padding=rates[2],
        dilation=rates[2],
    )

    self.image_pooling = torch.nn.Sequential(
        torch.nn.AdaptiveAvgPool2d(1),
        ASPPpart(
            in_channels,
            out_channels,
            kernel_size=1,
            stride=1,
            padding=0,
            dilation=1,),)

    self.conv_cat = ASPPpart(
        5 * out_channels,
        out_channels,
        kernel_size=1,
        stride=1,
        padding=0,
        dilation=1,)

def forward(self, x):
    # TODO: Implement ASPP properly instead of the following
    _, _, H, W = x.shape
    out_1x1 = self.conv_1x1(x)

    out_branch_first = self.conv_dilated_first(x)

    out_branch_second = self.conv_dilated_second(x)

    out_branch_third = self.conv_dilated_third(x)

    out_pooling = self.image_pooling(x)
    out_pooling_interpolated = F.interpolate(
        out_pooling, size=(H, W), mode="bilinear"
    )

    out_cat = torch.cat(
        (
            out_1x1,
            out_branch_first,
            out_branch_second,
            out_branch_third,
            out_pooling_interpolated,),1,)
    out = self.conv_cat(out_cat)

    return out
```

Problem 2. Branched Architecture

Table 3: comparison of joint, branch, and distillation architecture

	multi-task metrics↑	semantic segmentation metrics↑	depth metrics↓	training time (h)	GPU memory consumption	number of network parameters
Joint Architecture	66.93	85.412	18.482	**2h 17m 1s**	**45%**	**26.7M**
Branch Architecture	68.429	85.621	17.192	2h 39m 36s	50%	32.1M
Task Distillation Architecture	**68.536**	**85.695**	**17.159**	2h 56m 31s	55%	34.5M

As the table shown above, we can see the comparative results of the three architectures in terms of performance, training time, GPU memory consumption, and number of network parameters. Notably, all our training is conducted with NVIDIA GeForce RTX 2080 Ti with GPU memory of 11 GiB.

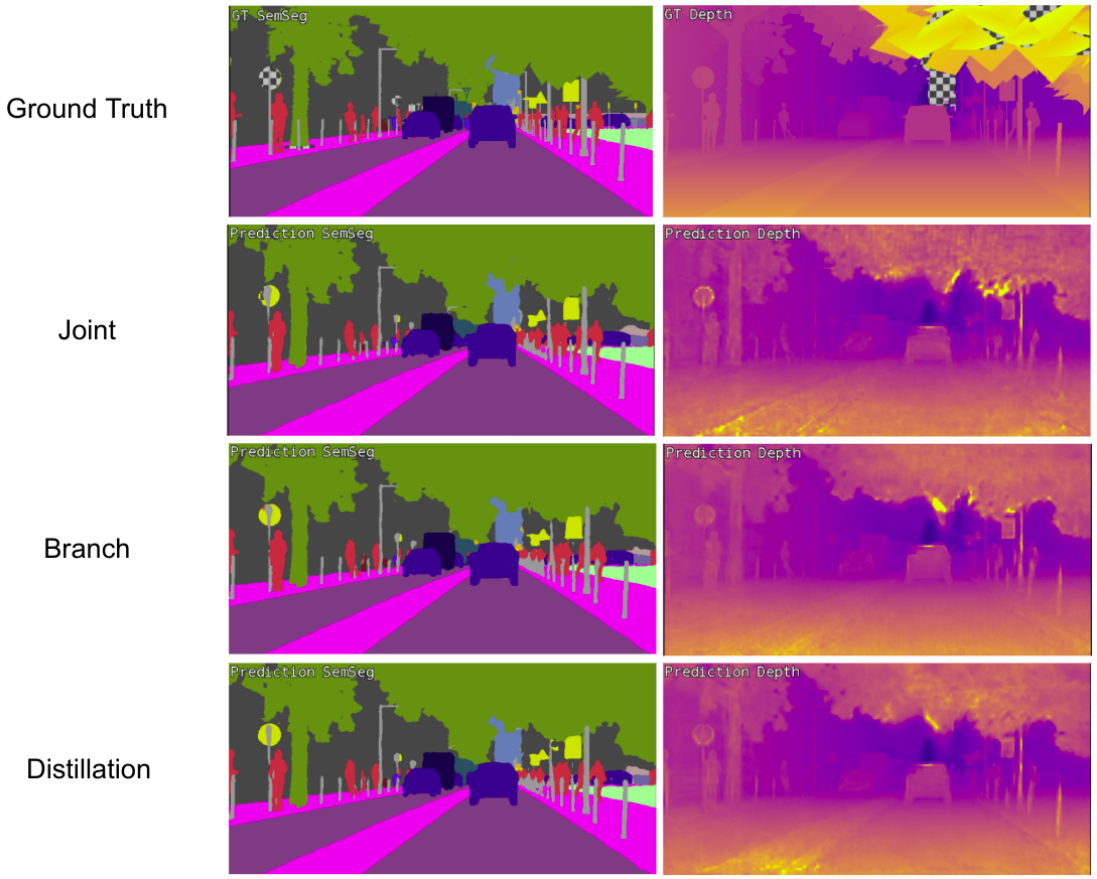

Figure 12: prediction results with different architectures

How does it compare to the joint architecture?

- **Performance**: The overall multi-task metrics increases substantially, which is mainly attributed to the improvement of depth metrics. In my opinion, the semantic segmentation task is basically a classification task, while depth estimation is a regression task. Thus, jointly computing them with one decoder may damage the performance because of the discrepancy of tasks.
- **Number of network parameters:** The number of network parameters grows from 26.7M to 32.1M. This is because we use task-specific ASPP modules and decoders compared to joint architecture, and double the parameters needed after the encoder.
- **GPU memory consumption**: As the number of parameters grows, we definitely need more space in the GPU to fit the model. Thus, the GPU memory consumption grows from 45% to 50%.
- **Training time:** As the number of parameters grows, the time to compute the gradients and update the parameters will certainly surge, which takes about 20 minutes more than the joint architecture.

Code snippets of main modifications is shown below.

```
# initialize separate ASPP modules for semantic segmentation and depth estimation
self.aspp_semseg = ASPP(ch_out_encoder_bottleneck, 256)
self.aspp_depth = ASPP(ch_out_encoder_bottleneck, 256)

# initialize separate decoder modules for semantic segmentation and depth estimation
self.decoder_semseg = DecoderDeeplabV3p(256, ch_out_encoder_4x, ch_out - 1)
self.decoder_depth = DecoderDeeplabV3p(256, ch_out_encoder_4x, 1)


# forward the lowest level features through ASPPs separately
features_tasks_1 = self.aspp_semseg(features_lowest)
features_tasks_2 = self.aspp_depth(features_lowest)

# decoding separately for semantic segmentation and depth estimation
predictions_4x_semseg, _ = self.decoder_semseg(features_tasks_1, features[4])
predictions_4x_depth, _ = self.decoder_depth(features_tasks_2, features[4])

# concatenate the results from the decoders above
predictions_4x = torch.cat((predictions_4x_semseg, predictions_4x_depth), 1)
```

Problem 3. Task Distillation

How does the distillation procedure compare to the branched architecture?

- **Performance**: The overall performance improves a little bit, which can be possibly attributed to the randomness in training, so we conclude that task distillation doesn't really make a difference in terms of multitask performance compared to branch architecture. One possible reason may be that it requires more epochs to fully utilize the capacity of this distillation architecture, since it's a slightly larger architecture compared to branch architecture, and 20 epochs may not be enough. Another reason may be simply summing up the output from the self attention module with the first decoder feature may be too naive, replacing it with a convolution operation may be more scientific in terms of combination.
- **Number of network parameters:** The number of network parameters grows from 32.1M to 34.5M, because of the introduction of a self-attention module and two more decoder modules.
- **GPU memory consumption:** It takes up 5% more GPU memory compared to the branch architecture. Firstly, it is because the task distillation model is larger than the branch model which can be instantiated by the number of network parameters shown above. Besides, there are actually two streams of gradients computed with respect to two sets of losses, and thus requires storing more gradient information in the GPU.
- **Training time:** It takes about 20 more minutes to train the distillation model compared to the branch model. Larger models with more parameters to update can be the main reason for longer training. Computing two sets of loss, and back propagating them through the whole network is also an important reason.

Code snippets of main modifications is shown below.

```
# initialization
self.aspp_semseg = ASPP(ch_out_encoder_bottleneck, 256)
self.aspp_depth = ASPP(ch_out_encoder_bottleneck, 256)

self.decoder_semseg_1 = DecoderDeeplabV3p(256, ch_out_encoder_4x, ch_out - 1)
self.decoder_depth_1 = DecoderDeeplabV3p(256, ch_out_encoder_4x, 1)

self.self_attention_semseg = SelfAttention(256, 256)
self.self_attention_depth = SelfAttention(256, 256)

self.decoder_semseg_2 = DecoderDistillation(256, ch_out - 1)
self.decoder_depth_2 = DecoderDistillation(256, 1)


# forwarding
features_tasks_1 = self.aspp_semseg(features_lowest)
features_tasks_2 = self.aspp_depth(features_lowest)
predictions_4x_semseg_1, features_4x_semseg_1 = self.decoder_semseg_1(features_tasks_1,
features[4])
predictions_4x_depth_1, features_4x_depth_1 = self.decoder_depth_1(features_tasks_2,
features[4])
features_4x_semseg_1_attend = self.self_attention_semseg(features_4x_semseg_1)
features_4x_depth_1_attend = self.self_attention_depth(features_4x_depth_1)

predictions_4x_semseg_2 = self.decoder_semseg_2(features_4x_semseg_1 +
features_4x_depth_1_attend)
predictions_4x_depth_2 = self.decoder_depth_2(features_4x_depth_1 +
features_4x_semseg_1_attend)
```

Problem 4. Open Challenge

In the open challenge we make several endeavors to improve the performance of the model.

DORN (Deep Ordinal Regression Network)[6]

In the original setting, we treat the depth estimation task as a regression problem and try to minimize its mean squared pixel loss. However, in DORN, we treat the problem as an ordinal regression problem.

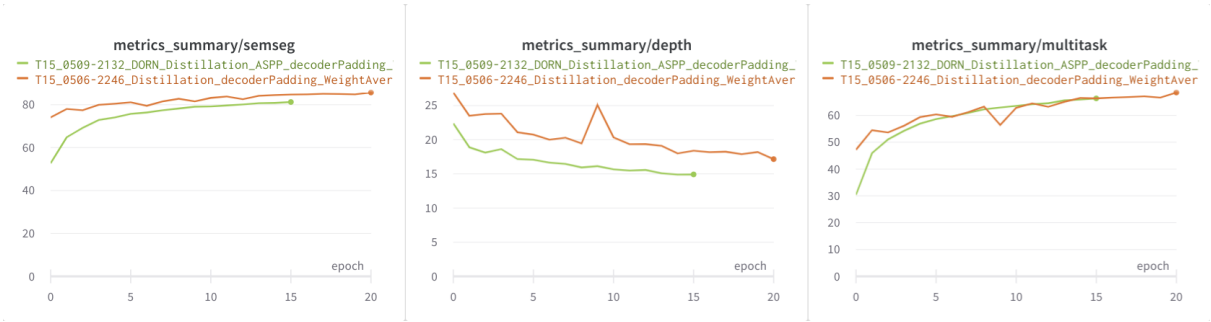

Figure 13: performance comparison of DORN

As shown in the plot above, after incorporating DORN into our distillation network, the depth metrics improves significantly, and even reach below 15 in 15 epochs, though the use of DORN seems to damage the semantic segmentation metrics. The training of the green curve was crashed, but the difference is obvious enough in 15 epochs.

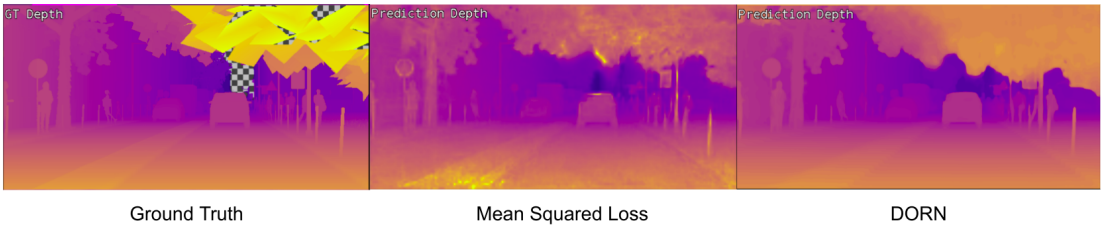

Figure 14: depth estimation results comparion of DORN

As we can see from the predicted depth using mean squared loss and DORN, the result of DORN is much smoother and closer to the ground truth visually. However, the result using mean squared loss is noisier as directly predicting depth for individual pixels loses correlation with the surrounding pixels.

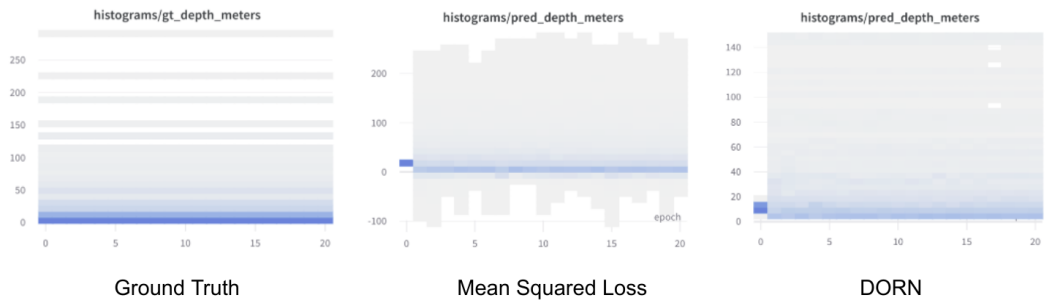

Figure 15: histograms of depth value distribution comparison

[6] Fu, Huan, et al. "Deep ordinal regression network for monocular depth estimation." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

Another possible reason for improvement is that DORN fits the real distribution of the depth better. In the original setting we simply normalize the depth value with its mean and variance, which assumes that the depth values are normally distributed. However, in DORN, we don't normalize the depth values before we forward them through the network, and the spacing-increasing discretization adapts to the condition that depth values tend to concentrate around the lower bound of the data range. As we can see from the figure above, the distribution of depth prediction with DORN emulates the ground truth better than using mean squared loss.

Code snippets of main modifications is shown below.

```python
class OrdinalRegressionLayer(nn.Module):
    def __init__(self):
        super(OrdinalRegressionLayer, self).__init__()

    def forward(self, x, mode=None):
        """
        :param x: NxCxHxW, N is batch_size, C is channels of features
        :return: ord_label is ordinal outputs for each spatial locations , N x 1 x H x W
                 ord prob is the probability of each label, N x OrdNum x H x W
        """
        N, C, H, W = x.size()
        ord_num = C // 2

        x = x.view(-1, 2, ord_num, H, W)
        if mode == "train":
            prob = F.log_softmax(x, dim=1).view(N, C, H, W)
            return prob

        ord_prob = F.softmax(x, dim=1)[:, 0, :, :, :]
        ord_label = torch.sum((ord_prob > 0.5), dim=1)
        return ord_prob, ord_label


class OrdinalRegressionLoss(object):
    def __init__(self, ord_num, beta, discretization="SID"):
        self.ord_num = ord_num
        self.beta = beta
        self.discretization = discretization

    def _create_ord_label(self, gt):
        N, H, W = gt.shape
        print("gt shape:", gt.shape)

        ord_c0 = torch.ones(N, self.ord_num, H, W).to(gt.device)
        if self.discretization == "SID":
            label = self.ord_num * torch.log(gt) / np.log(self.beta)
        else:
            label = self.ord_num * (gt - 1.0) / (self.beta - 1.0)
        label = label.long()
        mask = (
            torch.linspace(0, self.ord_num - 1, self.ord_num, requires_grad=False)
            .view(1, self.ord_num, 1, 1)
            .to(gt.device)
        )
        mask = mask.repeat(N, 1, H, W).contiguous().long()
        mask = mask > label.unsqueeze(1)
        ord_c0[mask] = 0
        ord_c1 = 1 - ord_c0

        ord_label = torch.cat((ord_c0, ord_c1), dim=1)
        return ord_label, mask
```

```
def __call__(self, prob, gt):
    """
    :param prob: ordinal regression probability, N x 2*Ord Num x H x W, torch.Tensor
    :param gt: depth ground truth, NXHxW, torch.Tensor
    :return: loss: loss value, torch.float
    """
    # N, C, H, W = prob.shape
    valid_mask = gt > 0.0
    ord_label, mask = self._create_ord_label(gt)
    # print("prob shape: {}, ord label shape: {}".format(prob.shape, ord_label.shape))
    entropy = -prob * ord_label
    loss = torch.sum(entropy, dim=1)[valid_mask]
    return loss.mean()
```

SE (Squeeze-and-Excitation)

SEresnet[7]:

In the original paper of SE, they show that by adding SE-blocks to ResNet-50 one can expect almost the same accuracy as ResNet-101 delivers. Therefore, we add SE-blocks to our backbone, and expect that a more powerful one will improve the multitask performance.

SEASPP[8]:

The idea of adding SE block after the concatenation of ASPP features is borrowed from [8]. As after concatenating the feature maps from different scales, the number of channels is very large, thus it's reasonable to use the SE block to weight its channels.

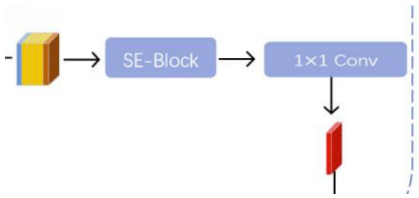

Figure 16: modification of ASPP with SE

Table 4: performance comparision of adding SE to different part of Branch architecture

	multi-task metrics↑	semantic segmentation metrics↑	depth metrics↓
Branch	68.429	85.621	17.192
SEresnet-Branch	68.514	85.651	17.137
SEASPP-Branch	68.685	85.722	17.037

[7] Hu, Jie, Li Shen, and Gang Sun. "Squeeze-and-excitation networks." Proceedings of the IEEE conference on computer vision and pattern recognition. 2018.

[8] Zhang, Kaihui, Xianhui Liu, and Yufei Chen. "Research on Semantic Segmentation of Portraits Based on Improved Deeplabv3+." IOP Conference Series: Materials Science and Engineering. Vol. 806. No. 1. IOP Publishing, 2020.

As shown in the table above, both adding SE to resnet and ASPP only slightly improve the performance.

Final Model: SEASPP + DORN

Finally we combine SEASPP and DORN to present our final model, which achieves 83.946 in semantic segmentation, **14.771** in depth estimation, **69.175** in multitask. (the training ran out of CUDA memory when loading the test data, thus didn't produce the submission file, but can be verified in wandb)